# Component-based Software Engineering

Soon Phei Tin

# Topic

- Part 1: Introduction to CBSE
- Part 2: Components in Spring Boot: Foundations
- Part 3: Component Interfaces, Composition, and Lifecycle
- Part 4: Advanced Component Features in Spring Boot
- Part 5: Component-Based Architecture with Spring Boot
- Part 6: Challenges, Best Practices, and Wrap-Up

# Part 1: Introduction to CBSE

- CBSE is the software engineering practices that developing software mainly based on software components.

- We will focus on three core principles: **Components**, **Interfaces**, and **Composition**

- Beyond the core principles, understanding CBSE requires a deeper look at **components**, **component models**, and **middleware**, as well as how they interrelate.

# Benefits and Challenges

**Benefits**

- Reusability reduces development time and costs.
- Modularity enhances maintainability and scalability.
- Easier testing and debugging of isolated components.

**Challenges**

- Dependency management and version conflicts.
- Overhead from inter-component communication.
- Designing components that are both reusable and specific enough for practical use.

# Components

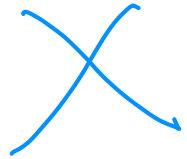
- In CBSE, a component is a modular, reusable, and independently deployable unit of software that encapsulates specific functionality and data.
- It provides services to other parts of the system through well-defined interfaces, hiding its internal implementation details.
- This modularity promotes reusability and maintainability.

# Components

Here are the summarized characteristics of components:

- **Self-contained:** Each component encapsulates its own logic and data.
- **Reusable:** Designed to be used across different systems or contexts.
- **Replaceable:** Can be swapped with another component that provides the same interface.
- **Interface-driven:** Interact with other components solely through defined interfaces, promoting loose coupling and high cohesion.

# Components in Spring Boot



- In Spring Boot, components are typically classes annotated with @Component, @Service, @Repository, or @Controller.

- These annotations tell the Spring Inversion of Control (IoC) container to manage the class, handling its instantiation, dependency injection, and lifecycle

- UserService is a self-contained component that provides a specific service (fetching user data).

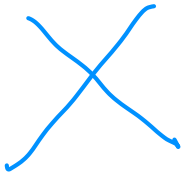
- Other parts of the application can use it without knowing how it retrieves the data, aligning with CBSE's emphasis on encapsulation and modularity.

```
@Service
public class UserService {
    public User getUserById(Long id) {
        // Logic to fetch a user by ID
        return new User(id, "John Doe");
    }
}
```

# Interfaces

- Interfaces in CBSE define the contract that components must follow.
- They specify what services a component provides (and sometimes requires) without exposing its internal workings.
- This abstraction enables loose coupling, as components can be swapped out as long as they adhere to the same interface.

# Interfaces in Spring Boot



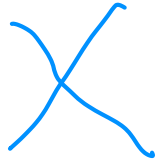
- In Spring, interfaces are commonly used to define service contracts, with concrete implementations injected at runtime.

- The UserRepository interface acts as a contract, allowing different implementations (e.g., JPA, JDBC, or in-memory) to be used interchangeably.

- This reflects CBSE's focus on well-defined interfaces for component interaction.

```java
public interface UserRepository {
    User findById(Long id);
}

@Repository
public class JpaUserRepository implements UserRepository {
    @Override
    public User findById(Long id) {
        // JPA-specific implementation to fetch user from database
        return new User(id, "Jane Doe");
    }
```

# Composition



- Composition in CBSE involves assembling components into a complete system by connecting them through their interfaces.

- This process leverages dependency relationships to create a functional whole, ensuring that components work together seamlessly.

# Composition in Spring Boot

- Spring Boot uses dependency injection (DI) to compose components.
- The UserService is composed with a UserRepository via constructor inject
- Spring's IoC container manages this composition, connecting the componer at runtime based on their interfaces.

```
@Service
public class UserService {
    private final UserRepository userRepository;

    @Autowired
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User getUserById(Long id) {
        return userRepository.findById(id);
    }
}
```

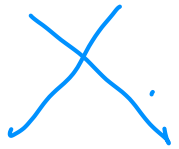
# Component Models

A component model is a framework or set of standards that governs how components are defined, implemented, and composed.

It provides:

- **Conventions:** Rules for creating components (e.g., how to specify interfaces).
- **Interaction Mechanisms:** How components communicate (e.g., method calls, events).
- **Lifecycle Management:** How components are instantiated, used, and destroyed.

# Component Models



- Examples of component models include
  - Enterprise JavaBeans (EJB)
  - Microsoft’s COM
  - Spring

- A component model ensures consistency and interoperability across a system.

# Spring Component Model

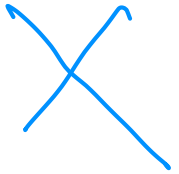
Relies on Plain Old Java Objects (POJOs) enhanced with annotations:

- @Component (and its derivatives) identifies components.

- @Autowired specifies dependencies.

- Spring Boot’s auto-configuration further simplifies this by automatically setting up components based on classpath dependencies.

# Component and Component Models



- The component model defines the “rules of the game” for components.

- Components are instances that conform to these rules, ensuring they can be managed and composed effectively within the system.

# Middleware

Middleware is software that sits between the operating system and applications, providing services that simplify component interaction and system integration.

It handles:

- **Communication:** Enabling components to talk to each other, even across networks (e.g., via RPC or messaging).
- **Data Management:** Abstracting database access or caching.
- **Infrastructure Services:** Offering security, transaction management, or load balancing.

# Middleware

In CBSE, middleware acts as the “glue” that supports component composition and execution, especially in distributed systems.

**Spring Example:**

- **Spring Data:** Acts as middleware by providing a consistent data access layer between components and databases.
- **Spring Cloud:** Offers middleware features like service discovery (Eureka), load balancing (Ribbon), and circuit breaking (Hystrix) for distributed Spring Boot applications.
- For instance, a Spring Boot app might use Spring Cloud to connect a UserService component on one server with a UserRepository component on another, abstracting the network complexity.

# Components, Component Models, and Middleware

- **Components and Component Models:** The component model defines how components are structured and interact. Components are the concrete realizations of this model.

- **Components and Middleware:** Middleware provides the runtime environment and services that components rely on to function. A component like UserService might use middleware (e.g., Spring Data) to access a database without handling the low-level details itself.

# Components, Component Models, and Middleware

- **Component Models and Middleware:** The component model often assumes a specific middleware environment. Spring's model, for instance, integrates tightly with its middleware (e.g., Spring Cloud), ensuring components can leverage distributed system features seamlessly.

Together, they form a layered system:

1. **Components** deliver customized functionality.
2. **Component Models** standardize their creation and composition.
3. **Middleware** enables their execution and interaction, especially in complex or distributed scenarios.

# Part 2: Components in Spring Boot: Foundations

**Objectives**

- Learn how Spring Boot implements CBSE principles.
- Understand component definition and basic dependency injection.

# Spring Boot Overview

Spring Boot is an extension of the Spring Framework that streamlines the development of production-ready applications by reducing configuration complexity and providing sensible defaults.

Key features include:

- **Auto-configuration**: Spring Boot automatically configures components based on the dependencies present in the classpath.
- **Embedded Servers**: Built-in support for web servers like Tomcat or Jetty allows applications to run without external server setup.
- **Opinionated Defaults**: Provides pre-configured settings for common tasks, which developers can override as needed.

# Spring Boot for CBSE

Spring Boot’s architecture is inherently component-based, making it an ideal framework for implementing CBSE principles:

- Components are managed by the Spring IoC container, ensuring modularity and reusability.
- Dependency injection facilitates loose coupling between components.
- Auto-configuration and starter dependencies simplify the integration and composition of components into cohesive applications.

# Components in Spring Boot

A **component** in Spring is any class that is managed by the Spring IoC container. These classes are typically annotated to indicate their role and enable automatic detection by Spring.

- **Core Annotations**:
  - **@Component**: Spring-managed bean.
  - **@Service**: Encapsulates business logic.
  - **@Repository**: Data access component.
  - **@Controller / @RestController**: Components for HTTP requests handling.
- **Component Registration**:
  Spring Boot uses a process called **component scanning** to automatically detect and register components. By default, Spring scans the package containing the main application class and all its sub-packages.

# Components in Spring Boot

- Example

```java
import org.springframework.stereotype.Service;

@Service
public class GreetingService {
    public String greet() {
        return "Hello, World!";
    }
}
```

- The @Service annotation marks GreetingService as a component.
- Spring Boot detects it during component scanning and registers it as a bean in the IoC container.

# Inversion of Control (IoC) Container 控制反转容器.

The Spring's component management system, embodying the **IoC** principle.

In IoC, the framework takes control of creating and managing objects, rather than the application code doing so manually.

- **Role of the IoC Container**:
  - Instantiates components (referred to as **beans**).
  - Configures them by injecting dependencies and setting properties.
  - Manages their lifecycle from creation to destruction.
  - Stores beans in the **application context**, which serves as the runtime environment.

# Dependency Injection (DI) 依赖注入

- **Dependency Injection** is a design pattern where a component's dependencies are provided by the framework rather than being instantiated within the component itself.

- This enhances modularity, testability, and maintainability.

- **Types of Dependency Injection**:
  - **Constructor Injection**: Recommended for mandatory dependencies.
  - **Setter Injection**: Suitable for optional dependencies or runtime changes.
  - **Field Injection**: Direct injection into fields (less preferred due to testability concerns)

# Constructor Injection

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class OrderService {
    private final PaymentService paymentService;

    @Autowired
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

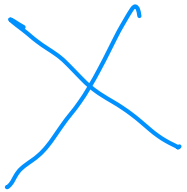
# Setter Injection

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class NotificationService {
    private EmailService emailService;

    @Autowired
    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }
}
```

# Field Injection

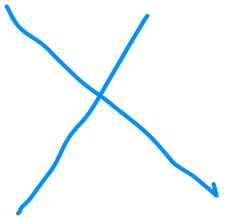
```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
}
```

# Advantages of DI



- **Advantages of DI**:
  - **Decoupling**: Components are independent of how their dependencies are created.
  - **Testability**: Dependencies can be mocked or stubbed in tests.
  - **Flexibility**: Dependencies can be swapped or reconfigured without altering component code.

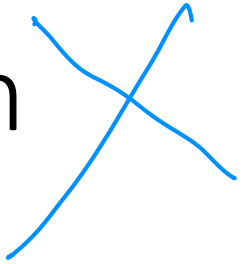
# Component Discovery and Customization



- Spring Boot’s components are discovered during component scanning, but it also offers customization options for fine-grained control.
- **Component Scanning**: By default, Spring scans the package of the main application class and its sub-packages.
- Customize this behavior with @ComponentScan:

```
@SpringBootApplication
@ComponentScan(basePackages = {"com.example.services", "com.example.repositories"}
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

# Component Discovery and Customization



- Define beans explicitly in a configuration class using @Bean
- This approach is useful for third-party classes or when more control is needed

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import com.zaxxer.hikari.HikariDataSource;

@Configuration
public class AppConfig {
    @Bean
    public DataSource dataSource() {
        return new HikariDataSource();
    }
}
```

# Practical Example: Building a Simple REST API

```
public class Product {
    private Long id;
    private String name;
    private double price;

    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}
```

# Practical Example: Building a Simple REST API

```java
import org.springframework.stereotype.Service;
import java.util.Arrays;
import java.util.List;

@Service
public class ProductService {
    public List<Product> getAllProducts() {
        // Hardcoded for simplicity
        return Arrays.asList(
            new Product(1L, "Laptop", 999.99),
            new Product(2L, "Smartphone", 499.99)
        );
    }
}
```

# Practical Example: Building a Simple REST API

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService productService;

    @Autowired
    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public List<Product> getProducts() {
        return productService.getAllProducts();
    }
```

# Practical Example: Building a Simple REST API

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ProductApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductApplication.class, args);
    }
}
```

# Part 3: Component Interfaces, Composition, and Lifecycle 生命周期

**Objectives**

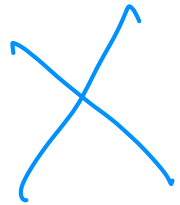
- Understand the use of interfaces for component abstraction.
- Explore component composition techniques and lifecycle management.

# Interfaces in CBSE and Spring Boot

Interfaces are a cornerstone of CBSE because they establish a **contract** that components must follow.

This contract specifies what services a component offers (its capabilities) and what it requires (its dependencies).

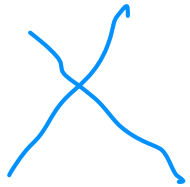
# Interfaces in CBSE and Spring Boot



By defining interactions through interfaces rather than concrete implementations, interfaces enable:

- **Polymorphism**: Different implementations of the same interface can be swapped seamlessly.
- **Loose Coupling**: Components depend on abstractions, reducing tight dependencies on specific implementations.
- **Testability**: Interfaces simplify mocking dependencies during unit testing.

# Spring Boot Example: Defining a Service Interface



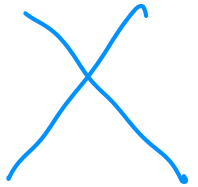
- The PaymentProcessor interface defines a contract for processing payments.
- CreditCardProcessor and PayPal Processor provide concrete implementations that can be used interchangeably.

```java
public interface PaymentProcessor {
    void processPayment(double amount);
}

@Service
public class CreditCardProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment of $" + amount);
    }
}

@Service
public class PayPalProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing PayPal payment of $" + amount);
    }
}
```

# Benefits of Using Interfaces

- **Flexibility**: Switch between payment processors (e.g., credit card to PayPal) by injecting a different implementation.
- **Extensibility**: Add new payment methods (e.g., CryptoProcessor) without altering existing code.
- **Testability**: Mock the PaymentProcessor interface in tests to isolate dependent components.

# Composition Techniques

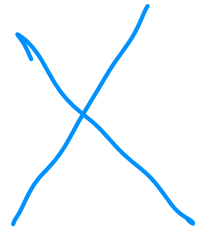


- In CBSE, **composition** refers to assembling components into a functional system by connecting them through their interfaces.

- Spring Boot achieves this primarily through **dependency injection (DI)**, a mechanism where the framework provides a component's dependencies automatically.

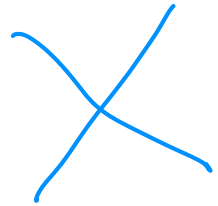
- Spring supports several DI techniques, each suited to different scenarios.

# Best Practice for Dependency Injection



- **Prefer Constructor Injection:** Use it for mandatory dependencies to ensure immutability and explicitness.
- **Use Setter Injection Sparingly:** Reserve it for optional or reconfigurable dependencies.
- **Best Practice for Dependency Injection** Opt for constructor or setter injection to improve testability and maintainability.

# Component Lifecycle Management



- Spring Boot’s Inversion of Control (IoC) container manages the entire lifecycle of components (beans), from instantiation to destruction.

- Understanding this lifecycle allows developers to perform initialization tasks (e.g., loading resources) and cleanup tasks (e.g., closing connections) at the appropriate stages.

# Key Phases of the Component Lifecycle

- **Creation**
  - The IoC container creates the bean instance.
  - Dependencies are injected based on the chosen DI method (constructor, setter, or field).
- **Initialization**
  - After dependency injection, Spring invokes initialization logic.
  - **@PostConstruct**: Annotate a method to run it after the bean is fully constructed.
- **Usage**
  - The bean is fully initialized and available for use by other components or services.
- **Destruction**
  - Before the application context closes, Spring invokes destruction logic.
  - **@PreDestroy**: Annotate a method to run it before the bean is destroyed.

```java
@Service
public class CacheService {
    private Map<String, Object> cache = new HashMap<>();

    @PostConstruct
    public void init() {
        System.out.println("CacheService initialized. Loading cache...");
        // Simulate loading initial cache data
        cache.put("defaultKey", "defaultValue");
    }

    public void put(String key, Object value) {
        cache.put(key, value);
    }

    public Object get(String key) {
        return cache.get(key);
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("CacheService shutting down. Clearing cache...");
        cache.clear();
    }
}
```

# Why Lifecycle Management Matters

- **Resource Initialization**: Set up connections, load configurations, or pre-populate data (e.g., cache).
- **Resource Cleanup**: Release resources like database connections, file handles, or memory to prevent leaks.
- **Controlled Behavior**: Ensure components start and stop gracefully, maintaining application stability.

# Best Practices for Component Design

- Define Interfaces for Abstraction
- Favor Constructor Injection
- Single Responsibility Principle
- Leverage Lifecycle Hooks (@PostConstruct and @PreDestroy)
- Avoid Circular Dependencies
- Document Interfaces (e.g., JavaDoc)

# Practical Example: Composing Components with Interfaces

**Scenario**

- An OrderService depends on a PaymentProcessor (interface with multiple implementations) and a NotificationService. The system processes payments and sends notifications when an order is placed.

1. Payment processor interface

```
public interface PaymentProcessor {
    void processPayment(double amount);
}
```

# Practical Example: Composing Components with Interface

- Implement concrete payment processors

```
@Service
public class CreditCardProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment of $" + amount);
    }
}

@Service
public class PayPalProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing PayPal payment of $" + amount);
    }
}
```

# Practical Example: Composing Components with Interface

- Define NotificationService

```
@Service
public class NotificationService {
    public void sendNotification(String message) {
        System.out.println("Sending notification: " + message);
    }
}
```

# Practical Example: Composing Components with Interface

- Compose the OrderService with dependencies

```
@Service
public class OrderService {
    private final PaymentProcessor paymentProcessor;
    private final NotificationService notificationService;

    @Autowired
    public OrderService(PaymentProcessor paymentProcessor, NotificationService
notificationService) {
        this.paymentProcessor = paymentProcessor;
        this.notificationService = notificationService;
    }

    public void placeOrder(double amount, String customerEmail) {
        paymentProcessor.processPayment(amount);
        notificationService.sendNotification("Order placed successfully for " +
customerEmail);
    }
}
```

# Practical Example: Composing Components with Interface

- Resolve multiple implementations
- Option 1: @Primary

```
@Service
@Primary
public class CreditCardProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment of $" + amount);
    }
}
```

# Practical Example: Composing Components with Interface

- Resolve multiple implementations
- Option 2: @Qualifier

```
@Service
public class OrderService {
    private final PaymentProcessor paymentProcessor;
    private final NotificationService notificationService;

    @Autowired
    public OrderService(@Qualifier("creditCardProcessor") PaymentProcessor
paymentProcessor,
                        NotificationService notificationService) {
        this.paymentProcessor = paymentProcessor;
        this.notificationService = notificationService;
    }

    // ... rest of the code
}
```